Skill gap

Készséghiány-elemző

1. A program célja

A felhasználó kiválaszt egy informatikai foglalkozást, a program pedig megállapítja, hogy **rendelkezik-e a szükséges készségekkel**, és ha nem, **melyek hiányoznak**.

- **Bemenet:** egy személy és egy célfoglalkozás (pl. *webfejlesztő*).
- **Kimenet:** a három lehetséges döntés egyike:
  - **READY:** minden lényeges készség megvan vagy lefedett;
  - **GAP:** legalább egy lényeges készségről a személy kijelentette, hogy *nem rendelkezik vele*;
  - **UNKNOWN:** nincs ismert hiány, de van olyan lényeges készség, amelyről még nem nyilatkozott.
- **Kiegészítés:** a 3 legkönnyebben elérhető **hasonló foglalkozás**, valamint a döntés teljes indoklása (melyik készség miért számít meglévőnek).

A foglalkozások és a készségek az **ESCO** európai osztályozásból származnak, a személyek adatai generáltak.

2. Tárolók és szerepük

Az adatok **két forrástárolóban** (PostgreSQL, MongoDB) vannak, ezekből **négy származtatott másolat** készül (Redis, Neo4j, pgvector, Qdrant).

Tároló	Tartalom	Indoklás
PostgreSQL	személyek, készségnyilatkozatok, foglalkozás- és készségazonosítók, döntésnapló az indoklással	Az üzleti entitások és a döntések idegen kulcsot, CHECK-feltételt és tranzakciót igényelnek.
MongoDB	az ESCO-dokumentumok eredeti formájukban (angol és magyar)	A dokumentumok szerkezete eltérő (alternatív nevek, megjegyzés és tágabb foglalkozás csak egy részükben van), a kötelező részt a `$jsonSchema` rögzíti.
Redis	a kész elemzés a `person:{id}:occupation:{id}:analysis` kulcson, 1 órás TTL-lel	Mind az öt tárolót érintő döntés gyors, lejáró gyorsítótára (cache-aside).
Neo4j	`Occupation-[:ESSENTIAL|OPTIONAL]->Skill`, `Skill-[:BROADER]->Skill`	Annak megállapítása, hogy egy készség alatt milyen meglévő készségek vannak a hierarchiában, változó hosszúságú útkeresés.
pgvector, Qdrant	965 foglalkozás- és készségleírás vektora (`all-MiniLM-L6-v2`, koszinusz)	Jelentés szerinti keresés szabad szövegre (pl. „hálózatokat szeretnék védeni”).

Forrás és származtatott adat:

- A **PostgreSQL** a személyek és a döntések forrása.
- A **MongoDB** az ESCO-dokumentumok forrása.
- A **Neo4j-csomópontok**, a **vektorok** és a **Redis-bejegyzések** tárolják a forrásazonosítót (`source_id`, Redisben `analysis_id`) és a legutóbbi frissítés idejét (`refreshed_at`).

3. Adatok

A foglalkozások és készségek valódi ESCO-adatok, a személyek és nyilatkozataik generáltak.

- **ESCO REST API**, letöltés dátuma: 2026-10-20.
  - Az ISCO-08 **25** (IKT-szakemberek) és **35** (IKT-technikusok) csoportjának minden foglalkozása: **87 foglalkozás**.
  - Az ezekhez kapcsolt összes készség: **878 készség**, köztük **256** tágabb-szűkebb kapcsolat.
  - Licenc: © Európai Unió, 2011/833/EU határozat.
- **Személyek:** 200 fő, Faker `hu_HU`, seed 42. Mindenkinek van jelenlegi foglalkozása (ennek lényeges készségei többnyire megvannak) és célfoglalkozása (a jelenlegivel leginkább átfedő 5 foglalkozás egyike).
- **Rekordszám tárolónként:**
  - MongoDB, pgvector, Qdrant, Neo4j: 965 dokumentum, vektor, illetve csomópont;
  - Redis: 200 elemzés;
  - PostgreSQL: 200 személy és **5 788 nyilatkozat** a kapcsolótáblában.

4. A döntés lépései

Minden tároló egy jól elkülönített kérdésre ad választ, a szabály pedig rögzített sorrendben dönt.

1. **PostgreSQL:** a személy nyilatkozatai (`has_skill` igaz vagy hamis; ha nincs sor, a személy nem nyilatkozott).
2. **MongoDB:** a célfoglalkozás lényeges készségei, közvetlenül **a forrásdokumentumból**.
3. **Neo4j:** minden lényeges készséghez a személy alatta lévő, meglévő készségei (`BROADER*` út).
4. **Készségállapot:**
   - `held`: a személy rendelkezik vele;
   - `lacking`: a személy kijelentette, hogy nem rendelkezik vele;
   - `covered`: nem nyilatkozott róla, de egy **szűkebb** készsége megvan (pl. Python, ami a programozás alá tartozik);
   - `not_stated`: nem nyilatkozott róla, és szűkebb készsége sincs.
   - A kifejezett „nem rendelkezik” nyilatkozat **elsőbbséget élvez** a meglévő szűkebb készséggel szemben.
5. **Szabály (`gap-v1`), ebben a sorrendben:** ha van `lacking`, a döntés **GAP**; ha van `not_stated`, **UNKNOWN**; egyébként **READY**.
6. **Vektoros keresés:** a célhoz legközelebbi 6 foglalkozás, mindegyik ugyanazzal a szabállyal elbírálva; közülük a 3 legmagasabb teljesítési arányú jelenik meg.

7. **Naplózás és gyorsítótár:** a PostgreSQL az indoklással együtt elmenti a döntést, a Redis gyorsítótárazza.

5. A fő lekérdezések

Neo4j: lefedés a készséghierarchiában

```
UNWIND $required_skill_ids AS skill_id
OPTIONAL MATCH (narrower:Skill)-[:BROADER*1..]->(:Skill {id: skill_id})
WHERE narrower.id IN $held_skill_ids
RETURN skill_id, collect(DISTINCT narrower.id) AS narrower_held_skill_ids
```

pgvector: egyetlen SQL-utasítás, az üzleti táblákkal összekapcsolva

```
SELECT s.id, s.label, 1 - (c.embedding <=> $vector) AS score, ps.has_skill
FROM concept_embeddings c
JOIN skills s ON s.id = c.concept_id
LEFT JOIN person_skills ps ON ps.skill_id = s.id AND ps.person_id =
$person_id
WHERE c.kind = 'skill'
ORDER BY c.embedding <=> $vector LIMIT 10;
```

Ugyanez **Qdranttal két lépésben** történik: vektoros keresés a Qdrantban, majd a nyilatkozatok lekérése PostgreSQL-ből és az eredmények összefésülése Pythonban, mivel a Qdrant nem fér hozzá a `person_skills` táblához.

MongoDB: validáció. `$jsonSchema`, `validationLevel: "strict"`, `validationAction: "error"`. Például a lényeges készség nélküli foglalkozást vagy a `skill_type: "tool"` értéket az adatbázis hibával elutasítja.

Redis: cache-aside. A program először a kulcsot olvassa; ha az nem létezik, elvégzi a teljes elemzést, naplózza, majd 1 órás TTL-lel eltárolja.

6. Konzisztencia

Ha a forrás megváltozik, a másolatok elavulnak, és a frissítési útvonal hozza őket újra összhangba.

Egy változás több tárolón keresztül (`cli.py update-demo`):

Lépés	MongoDB	Redis	Neo4j	pgvector, Qdrant
1. kiindulás	eredeti dokumentum	READY	régi élek	régi vektor
2. dokumentum módosítva, frissítés **nélkül**	+1 lényeges készség	**még READY (elavult)**	**még régi**	**még régi**
3. frissítési útvonal	változatlan	törölve,	új	új vektor, új

Lépés	MongoDB	Redis	Neo4j	pgvector, Qdrant
		újraszámolva: UNKNOWN	`ESSENTIAL` él	`refreshed_at`

7. Mérések

A teszteseteket a paraméterek beállítása előtt rögzítettük. Minden alábbi számot a `cli.py measure` parancs állít elő.

Tesztesetek (`data/tests/`):

- **30 döntési eset** (10 READY, 10 GAP, 10 UNKNOWN), a szabály szövege alapján címkézve. Szerepelnek köztük a hierarchia esetei is: a meglévő szűkebb készség lefedi a tágabbat, fordítva nem, és a kifejezett „nem rendelkezik” erősebb.
- **20 keresési lekérdezés:** mindegyik egy foglalkozás ESCO alternatív neve, amely nem tartalmazza a foglalkozás fő nevét. Az alternatív nevek ezért **nem** részei a vektorrá alakított szövegnek.

Eredmények:

	Qdrant	pgvector
döntési pontosság, 30 eset (hiányzó és lefedett készséglisták egyezése)	1,0 (1,0 / 1,0)	1,0 (1,0 / 1,0)
foglalkozás recall@5 / MRR@10, 20 lekérdezés	0,95 / 0,91	0,95 / 0,91
ANN recall@10, szűrő nélkül	1,0	1,0
ISCO 251-es szűrő (a sorok 4,6%-a): találatszám, recall@10	10; 0,985 és 1,0 között	10; 0,995 és 1,0 között
átlagos késleltetés szűrővel	kb. 51 ms	kb. 0,5 ms
beállítás	`hnsw_ef` 64; payload index: `concept_id`, `kind`, `isco_minor_group`	`hnsw.ef_search` 40; `iterative_scan strict_order`

- A két rendszer top-10 listája **mind a 20** lekérdezésnél ugyanazt a 10 azonosítót tartalmazza.
- **Az index használata:** 965 vektornál alapértelmezés szerint egyik rendszer sem használná a HNSW-indexet. Ezért a Qdrant indexelési küszöbe alacsonyabbra van állítva, a pgvector lekérdezéstervezője pedig az index használatára van kényszerítve, így a mérés valóban a HNSW-t vizsgálja.
- A szűrt recall két `seed` futtatás között **0,985 és 1,0 között változik**, mert mindkét rendszer véletlenszerűen osztja ki a HNSW-szinteket; egy `seed` után az eredmény megismételhető.

- **Késleltetés:** a Qdrant mérése a HTTP-hívást is tartalmazza, ezért kb. 51 ms; feltételezhető, hogy ez a fix költség elfedi a keresés idejét. A két rendszer késleltetése ezért közvetlenül nem összevethető.

8. Hibaesetek

Egy keresési és egy döntési hibaeset; mindkettő az adatok sajátosságából ered, nem programhibából.

Keresés: „ICT security tester"

- Ez az *ethical hacker* (etikus hacker) foglalkozás ESCO alternatív neve.
- Az első három találat: *ICT system tester*, *ICT security administrator*, *ICT integration tester*.
- **Ok:** az *ethical hacker* leírása sérülékenységvizsgálatról és behatolásos tesztelésről szól, az „ICT … tester" szöveg viszont a tesztelő foglalkozások nevéhez és leírásához áll közelebb. A modell nem ismeri fel, hogy a biztonsági tesztelő ebben az esetben etikus hackert jelent.
- **Lehetséges javítás:** az alternatív nevek bevonása a vektorrá alakított szövegbe. Ekkor azonban a tesztlekérdezések bekerülnének az indexbe, ezért új, független tesztkészletre lenne szükség.

Döntés: a hierarchia az ESCO-é, nem a tudás mélységét méri

- Egy hálózati architekt esetében a *create flowchart diagram* (folyamatábra készítése) megléte lefedi a *design process* (folyamattervezés) készséget, mert az ESCO az elsőt a második alá sorolja.
- **Kezelés:** az elemzés megjeleníti a következtetést („via create flowchart diagram"), így a felhasználó látja, és közvetlenül is nyilatkozhat a készségről.

9. Futtatás

```
docker compose up -d --build
docker compose exec app python cli.py seed              # minden tároló törlése és újratöltése
docker compose exec app python cli.py measure           # a 7. fejezet összes mérése
docker compose exec app python cli.py update-demo       # 6. fejezet
```

Webes felület: http://127.0.0.1:8001 (foglalkozások, elemzés, saját készségek, előzmények, adatbázis-böngésző).